HRT C++ — 45-Minute Technical Round

Deep-Dive Non-Coding Interview Preparation

30 Expert Questions — Full Answers + Code + Follow-ups + HRT Context

Coverage Map

- Topic 1: Object Lifetime — construction order, most vexing parse, delegating constructors, initialisation forms
- Topic 2: Templates Deep — argument deduction, specialisation vs overloading, variadic templates, if constexpr
- Topic 3: Memory Model & UB — strict aliasing, signed overflow, as-if rule, sequenced-before, OOTA values
- Topic 4: STL Internals — vector growth, deque layout, hash collisions, iterator invalidation rules
- Topic 5: Concurrency Deep — mutex types, ABA problem, happens-before chain, false sharing measurement
- Topic 6: Design & Systems — Pimpl, optional/variant/any, expression templates, intrusive containers, latency measurement

TOPIC 1 — Object Lifetime, Construction & Destruction

Q1. Walk me through the exact order of construction and destruction for a class with base classes, member variables, and virtual functions. [MO]

Answer

Construction order: (1) Virtual base classes (most-derived class constructs them, most-base first). (2) Non-virtual base classes, left-to-right in declaration order. (3) Member variables, in declaration order (NOT initializer-list order). (4) Constructor body. Destruction is exactly reverse. Critically: during construction, the virtual table is partially formed — calling a virtual function in a base constructor calls the BASE version, not the override. This is one of the most common C++ bugs.

Code

```
struct Base {
    Base()  { print(); }   // virtual dispatch does NOT reach Derived
    virtual void print() const { std::cout << "Base\n"; }
};

struct Derived : Base {
    int x = 42;
    Derived() : Base() {}  // Base() called first, x not yet set
    void print() const override { std::cout << "Derived x=" << x << "\n"; }
};

Derived d;
// Prints: "Base"  (NOT "Derived x=42")
// vptr points to Base::vtable during Base() execution

// Member declaration order matters, NOT initializer order:
struct Wrong {
    int b;
    int a;
    Wrong() : a(1), b(a) {} // BUG! b initialised first (decl order)
    // b uses uninitialised a -> UB
};

// FIX: declare a before b, or use body initialisation
struct Fixed {
    int a;  // declared first
    int b;
    Fixed() : a(1), b(a) {} // OK: a is initialised before b
};
```

Follow-up HRT may ask: What happens if a constructor throws halfway through? Which destructors run?

HRT Insight: HRT constructors for order/trade objects must be fully initialized or throw — half-constructed objects on the hot path silently corrupt state.

Q2. What is the "most vexing parse" and how does it affect object construction? [MO]

Answer

The most vexing parse (Scott Meyers term): C++ grammar ambiguity where the compiler interprets a declaration as a function prototype instead of an object. "Widget w();" declares a function returning Widget, not a Widget object. Any expression that "could be" a function declaration IS parsed as one. C++11 uniform initialisation (braces) largely solves this, but mixing () and {} has subtleties with initializer_list constructors.

Code

```
// MOST VEXING PARSE:
Widget w1();          // Declares a FUNCTION, not a Widget!
Widget w2(Widget());  // Also a function declaration!
```

```
// FIX: use braces (C++11 uniform initialisation)
Widget w3{};          // Object, zero-initialised
Widget w4{Widget{}};  // Object containing a Widget

// But braces have their own trap with initializer_list:
std::vector<int> v1(3, 5); // 3 elements, each = 5: {5,5,5}
std::vector<int> v2{3, 5}; // 2 elements: {3,5}
// Braces prefer initializer_list constructor!

// std::make_shared avoids the parse entirely:
auto w = std::make_shared<Widget>(); // unambiguous

// Rule for HRT code:
// - Named objects: Widget w{args};
// - Factories: std::make_unique<T>(args)
// - Never: T x(); unless declaring a function intentionally
```

Follow-up HRT may ask: How does the compiler choose between a constructor taking int and one taking initializer_list<int>?

HRT Insight: In trading system constructors for configuration objects, ambiguous parses cause silent incorrect defaults — brace-init consistently prevents this.

Q3. Explain delegating constructors (C++11). What are their benefits and pitfalls? [MO]

Answer

A delegating constructor calls another constructor of the same class as its first action. Benefits: eliminates code duplication across constructors, ensures invariants are established in one place. Pitfall: the object is considered "fully constructed" after the delegated-to constructor completes — so if the delegating constructor's body throws, the destructor WILL run (unlike non-delegating constructors that throw before completing). Also, you cannot initialise members in the initialiser list of a delegating constructor.

Code

```
class Order {
    int64_t price_;
    int32_t qty_;
    char    side_;
    void validate() {
        if (qty_ <= 0)  throw std::invalid_argument("qty must be > 0");
        if (price_<= 0) throw std::invalid_argument("price must be > 0");
    }
public:
    // Primary constructor: all validation here
    Order(int64_t price, int32_t qty, char side)
        : price_(price), qty_(qty), side_(side)
    { validate(); }

    // Delegating: reuses validation
    Order(int64_t price, int32_t qty)
        : Order(price, qty, 'B') {}  // delegates to primary

    // Delegating buy at market
    explicit Order(int32_t qty)
        : Order(0, qty, 'B') {}

    // ILLEGAL: cannot mix delegation and member init
    // Order(int64_t p) : Order(p, 100), price_(p) {} // COMPILE ERROR
};

Order o1(10250, 100, 'S'); // direct
Order o2(10250, 100);      // delegates -> primary
Order o3(50);              // delegates -> Order(0,50,B)
```

Follow-up HRT may ask: How does this interact with inheritance? Can you delegate to a base constructor?

HRT Insight: HRT uses delegating constructors for Order/Instrument objects to guarantee that validation (price > 0, qty > 0, valid symbol) always runs regardless of which constructor overload is called.

Q4. What is the difference between direct initialisation, copy initialisation, and list initialisation? When does each cause an implicit conversion? [MO]

Answer

Direct initialisation (T x(expr)): considers all constructors including explicit ones. Copy initialisation (T x = expr): only considers non-explicit constructors for implicit conversion; explicit constructors require a cast. List initialisation (T x{expr}): prohibits narrowing conversions (float->int, int->char etc.) — a compile error rather than silent truncation. This makes brace-init safer for numeric types.

Code

```
struct Widget {
    explicit Widget(int n) : n_(n) {}
    int n_;
};

Widget w1(42);      // OK: direct, explicit ctor allowed
// Widget w2 = 42;  // ERROR: copy-init, explicit ctor forbidden
Widget w3 = Widget(42); // OK: explicit cast then copy/move
Widget w4{42};      // OK: list-init, explicit allowed here

// Narrowing prohibition with braces:
int   i = 3.14;     // OK: silent truncation to 3 (bad!)
int   j{3.14};      // COMPILE ERROR: narrowing float->int

// double->float may be narrowing:
float f{3.14};      // WARNING/ERROR: double literal to float
float g{3.14f};     // OK: float literal, no narrowing

// Practical rule for trading price types:
int64_t price{getSomeDouble()};  // error if narrowing -- safe!
int64_t price2(getSomeDouble()); // silently truncates -- dangerous!
```

Follow-up HRT may ask: What is aggregate initialisation and when does it apply?

HRT Insight: HRT uses fixed-point int64_t for prices. Brace-init prevents accidental double-to-int64 narrowing of a floating-point price — a class of bug that can cause bad orders.

TOPIC 2 — Templates & Generic Programming (Deep)

Q5. What is template argument deduction? Explain the deduction rules for T, T&, T&&, and const T&. [Tmp]

Answer

Template argument deduction infers T from function call arguments. Key rules: for T (by value): top-level cv-qualifiers and references stripped — T deduces to the decayed type. For const T& (by const ref): T deduces without const, the reference binds to any lvalue or rvalue. For T& (by ref): T keeps cv-qualifiers, cannot bind to rvalues. For T&& (forwarding reference): T deduces to X& for lvalue X (reference collapsing gives X& &&= X&), and X for rvalue X. std::forward then uses this to reconstruct the original value category.

Code

```
template<typename T> void byVal(T x);        // T = decayed type
template<typename T> void byRef(T& x);       // T keeps cv-qual
template<typename T> void byCRef(const T& x);// T strips const
template<typename T> void byFwd(T&& x);      // forwarding reference

int         i  = 5;
const int   ci = 5;
int&&       ri = 5;

byVal(ci);    // T = int   (const stripped for by-value)
byVal(i);     // T = int
byRef(ci);    // T = const int  (cv preserved for ref)
// byRef(5);  // ERROR: rvalue cannot bind to T&
byCRef(5);    // T = int   (rvalue OK, const ref extends lifetime)

// Forwarding reference deduction:
byFwd(i);     // T = int&  -> T&& = int& && = int& (collapses)
byFwd(5);     // T = int   -> T&& = int&&
byFwd(ci);    // T = const int& -> T&& = const int&

// auto follows same rules:
auto  a = ci;  // int   (top-level const stripped)
auto& b = ci;  // const int& (const preserved)
auto&& c = ci; // const int& (forwarding ref)
auto&& d = 42; // int&&
```

Follow-up HRT may ask: What is std::decay_t and when do you need it explicitly?

HRT Insight: HRT code uses forwarding references extensively in message passing and pool allocator APIs — deduction errors here cause silent copies of heavy objects.

Q6. What is template specialisation vs overloading? Explain full and partial specialisation, and the pitfall of partially specialising function templates. [Tmp]

Answer

Function template overloading: add a new template with different constraints — selected by overload resolution. Full specialisation: provide a concrete implementation for specific types — selected after overload resolution. Partial specialisation: only allowed for class/variable templates, not function templates. The pitfall: adding a function template specialisation does not participate in overload resolution the way you expect — it is selected only when the primary template is chosen first. This is Herb Sutter's "Why Not Specialize Function Templates?" problem.

Code

```
// CLASS template partial specialisation: allowed
template<typename T>         struct Serialize { };
template<typename T>         struct Serialize<T*> { };// partial OK
template<>                   struct Serialize<int> { };// full OK

// FUNCTION template: NO partial specialisation
// Workaround 1: overloading (preferred)
template<typename T> void process(T x)     { std::cout<<"generic"; }
template<typename T> void process(T* x)    { std::cout<<"pointer"; }

// Workaround 2: delegate to class
template<typename T>
struct ProcessImpl { static void run(T) { std::cout<<"generic"; } };
template<typename T>
struct ProcessImpl<T*>{ static void run(T*){ std::cout<<"pointer"; } };

template<typename T>
void process2(T x) { ProcessImpl<T>::run(x); }
```

```
// THE PITFALL:
template<typename T>    void f(T)  { std::cout<<"primary"; }
template<typename T>    void f(T*) { std::cout<<"pointer overload"; }
template<>              void f(int*){ std::cout<<"int* spec"; }
// f(new int) -> "int* spec"? NO! -> "pointer overload"
// Specialisation of PRIMARY, not of the overload!
```

Follow-up HRT may ask: When would you use if constexpr instead of template specialisation?

HRT Insight: HRT serialisation and message encoding infrastructure uses class template specialisation heavily — understanding the pitfall prevents subtle encoding errors for pointer-like types.

Q7. What are variadic templates? Implement a type-safe printf-like logging function. [Tmp]

Answer

Variadic templates accept any number of type parameters (typename... Args). Parameter pack expansion uses ... in different positions: sizeof...(Args) for count, Args&&... for forwarding, (expr, ...) for fold expressions (C++17). They underlie std::tuple, std::variant, std::make_unique, and any heterogeneous container. Fold expressions (C++17) eliminate recursive unpacking for simple operations.

Code

```
// C++17 fold expression: sum any number of values
template<typename... Ts>
auto sum(Ts&&... args) {
    return (std::forward<Ts>(args) + ...); // unary right fold
}
sum(1, 2.0, 3L); // 6.0 (double)

// Type-safe log: each arg checked at compile time
template<typename... Args>
void logMsg(const char* fmt, Args&&... args) {
    // Compile-time: count format specifiers == sizeof...(Args)
    constexpr size_t nArgs = sizeof...(Args);
    // Runtime: format using index_sequence trick
    ([&](auto&& a) {
        while (*fmt && *fmt != '%') std::cout << *fmt++;
        if (*fmt == '%') { ++fmt; std::cout << a; }
    }(std::forward<Args>(args)), ...);
    while (*fmt) std::cout << *fmt++;
}

logMsg("price=% qty=% side=%", 10250, 100, 'B');
// "price=10250 qty=100 side=B"

// Practical: tuple construction via pack expansion
template<typename... Ts>
struct Packet {
    std::tuple<Ts...> fields;
    explicit Packet(Ts&&... ts)
        : fields(std::forward<Ts>(ts)...) {}
};
Packet p{10250LL, 100, 'B'};
```

Follow-up HRT may ask: How do you iterate over a parameter pack without recursion in C++17?

HRT Insight: HRT uses variadic templates for compile-time message schemas — each protocol message type is a Packet<Fields...> with zero runtime overhead.

Q8. What is if constexpr (C++17)? How does it differ from ordinary if and enable_if? [Tmp]

Answer

if constexpr evaluates its condition at compile time: the not-taken branch is not instantiated (only parsed). Unlike ordinary if: both branches of a normal if are fully instantiated — template code in the not-taken branch still causes a compile error if it is invalid for the type. Unlike enable_if: if constexpr is far more readable, does not require two overloads, and can appear anywhere inside a function body.

Code

```
// PROBLEM with regular if in templates:
template<typename T>
void process(T val) {
    if (std::is_integral_v<T>)
        val.decimal();  // ERROR even for int=true branch!
    // Both branches instantiated, decimal() fails for non-float T
}

// if constexpr: only taken branch is instantiated
template<typename T>
void process2(T val) {
    if constexpr (std::is_integral_v<T>) {
        std::cout << "int: " << val;
    } else if constexpr (std::is_floating_point_v<T>) {
        std::cout << "float: " << std::fixed << val;
    } else {
        val.customDisplay(); // only compiled if neither above
    }
}

// vs enable_if: two separate overloads (verbose)
template<typename T, std::enable_if_t<std::is_integral_v<T>,int>=0>
void processOld(T val) { std::cout << "int: " << val; }

template<typename T, std::enable_if_t<!std::is_integral_v<T>,int>=0>
void processOld(T val) { std::cout << "other: " << val; }

// if constexpr in trading: dispatch on message type at compile time
template<typename Msg>
void encode(const Msg& m, std::byte* buf) {
    if constexpr (std::is_trivially_copyable_v<Msg>)
        std::memcpy(buf, &m, sizeof(Msg));
    else
        m.serialize(buf);
}
```

Follow-up HRT may ask: Can if constexpr be nested? What about inside a lambda?

HRT Insight: HRT message encoding uses if constexpr to select between memcpy (trivial types) and custom serialisation (complex types) with zero runtime branch.

TOPIC 3 — Memory Model, UB & Low-Level Semantics

Q9. What is strict aliasing and why does violating it cause undefined behaviour even when the code "works" in debug builds? [UB]

Answer

The strict aliasing rule: you may only access an object through a pointer/reference of its own type, a base class type, or char/unsigned char/std::byte. Accessing through any other pointer type is UB. The compiler uses this rule to assume that pointers to different types cannot alias — enabling aggressive load/store reordering and elimination. In debug builds the optimiser is off so the reordering doesn't happen; in optimised builds it can silently break. The only safe way to reinterpret bytes is memcpy, std::bit_cast (C++20), or via char*.

Code

```
// STRICT ALIASING VIOLATION: UB in optimised builds
float f = 3.14f;
uint32_t bits = *reinterpret_cast<uint32_t*>(&f); // UB!
// Compiler: float* and uint32_t* cannot alias
// -> may eliminate the load of f entirely

// CORRECT: memcpy (compiler optimises to a register move)
uint32_t bits2;
std::memcpy(&bits2, &f, sizeof(f));  // defined, and fast

// CORRECT C++20: std::bit_cast
uint32_t bits3 = std::bit_cast<uint32_t>(f); // defined

// CORRECT: char/std::byte are exempt from strict aliasing
std::byte* raw = reinterpret_cast<std::byte*>(&f);
for (size_t i = 0; i < sizeof(f); ++i) std::cout << (int)raw[i];

// Common trading violation: casting network buffer to struct
// BAD:
struct Msg { uint32_t price; uint16_t qty; };
char buf[1500];
recvfrom(sock, buf, sizeof(buf), 0, nullptr, nullptr);
Msg* m = reinterpret_cast<Msg*>(buf); // UB! char* -> Msg*

// CORRECT:
Msg m2; std::memcpy(&m2, buf, sizeof(m2)); // defined
// Or: use __attribute__((packed)) struct + memcpy
```

Follow-up HRT may ask: What compiler flag disables strict aliasing optimisations? Why should you not rely on it in production?

HRT Insight: HRT's network parsers must comply with strict aliasing — violations cause silent corruption at -O3 that does not reproduce at -O0, making them nearly impossible to debug in production.

Q10. Explain signed integer overflow UB. How does it affect compiler optimisations, and what are the safe alternatives? [UB]

Answer

Signed integer overflow is UB in C++. The compiler uses the no-overflow assumption to optimise: it can assume (x+1 > x) is always true for signed int — removing bounds checks. It can simplify (i*2/2 == i) to true. It can turn (x < x+1) into an unconditional branch. Unsigned overflow is fully defined (wraps modulo 2^N). Safe alternatives: use unsigned arithmetic where wrapping is acceptable, use compiler builtins (__builtin_add_overflow), or use std::numeric_limits with pre-flight checks.

Code

```
// COMPILER EXPLOITS SIGNED OVERFLOW UB:
// Compiler may optimise this loop to run forever
// (assumes i never overflows, so i >= 0 always)
for (int i = 0; i >= 0; i++) { // infinite loop optimised!
    arr[i] = i;
}

// Bounds check elimination:
bool inRange(int x) { return x+100 > x; } // compiled to "return true"

// SAFE: unsigned arithmetic (wrapping is defined)
uint32_t u = UINT_MAX;
u++;  // wraps to 0: defined behaviour

// SAFE: compiler builtin
int result;
if (__builtin_add_overflow(a, b, &result))
```

```
    throw std::overflow_error("add overflow");

// SAFE: C++20 std::bit_cast friendly pattern
// Use int64_t to extend range:
int64_t safeCalc(int32_t a, int32_t b) {
    return (int64_t)a * b; // widen before multiply
}

// Trading: price * qty can overflow int32_t!
// price=150000, qty=10000 -> 1.5B > INT_MAX (2.1B) -- OK
// price=250000, qty=10000 -> 2.5B > INT_MAX! OVERFLOW!
int64_t notional = (int64_t)price * qty; // always use int64_t
```

Follow-up HRT may ask: What is the difference between -fwrapv, -ftrapv, and -fsanitize=undefined?

HRT Insight: HRT computes notional value (price * quantity) frequently. Signed overflow in price*qty silently produces a wrong (negative) notional — a bug worth real money.

Q11. What is the "as-if" rule and what limits does it place on compiler transformations? [UB]

Answer

The "as-if" rule: the compiler may transform code in any way as long as the observable behaviour (reads/writes to volatile objects, I/O operations, atomic operations with seq_cst) remains identical. This permits: reordering loads/stores, eliminating redundant computations, hoisting loop-invariant code out of loops, constant-folding, and inlining. It does NOT permit: changing the order of side effects that have defined sequencing, reordering volatile accesses, or changing atomic seq_cst operations. This is why non-atomic reads/writes can be reordered — they are not "observable" in the as-if sense.

Code

```
// AS-IF permits: hoist invariant computation
int sum = 0;
for (int i = 0; i < n; i++) {
    sum += expensive() * i; // compiler may call expensive() once
}

// AS-IF permits: eliminate dead stores
int x = 0;
x = 1;    // dead store: eliminated
x = 2;    // only this write survives

// AS-IF CANNOT: reorder volatile accesses
volatile int* reg = (volatile int*)0x40001000;
*reg = 1;  // must execute in program order
*reg = 2;  // must follow the first write

// AS-IF CANNOT: reorder seq_cst atomics
std::atomic<int> a{0}, b{0};
a.store(1); // seq_cst: cannot be moved after b.store
b.store(1); // both stores observed in this order globally

// CONSEQUENCE for trading:
int price = getPrice();  // non-volatile, non-atomic
logPrice(price);         // compiler may reorder with above
// If getPrice() has no side effects visible to compiler,
// it may be called after logPrice or even eliminated
```

Follow-up HRT may ask: How does the as-if rule interact with exception handling? Can a throw be reordered?

HRT Insight: The as-if rule explains why timestamp() logs in non-atomic trading code may produce timestamps that are out-of-order with the events they measure — the compiler legally reorders the reads.

Q12. What is the sequenced-before relation and how does it interact with function call order? [MM]

Answer

Sequenced-before is the formal within-a-thread ordering relation. Key rules: (1) Within a full expression, left-to-right evaluation order is NOT guaranteed except for &&, ||, ?:, and , operators. (2) Function arguments are evaluated in an unspecified order. (3) Postfix ++ has a sequenced-before relation only with its value computation, not with its side effect in the same expression. C++17 tightened some rules: in a<<b, a is sequenced before b; in f(a), f is evaluated before arguments.

Code

```
// UNSPECIFIED ORDER (pre-C++17):
int i = 0;
f(i++, i++); // UB! order of i++ side effects unspecified

// STILL unspecified in C++17 (argument evaluation order):
f(g(), h()); // g() and h() may run in any order

// C++17 GUARANTEED: a evaluated before b in a<<b
std::cout << f() << g();
// f() evaluated BEFORE g() (left-to-right in <<)

// C++17 GUARANTEED: function before its arguments
auto& func = getFuncRef();
func(modifiesFunc()); // getFuncRef() runs first

// SAFE: separate expressions
int a = i++; // i becomes i+1
int b = i++; // uses new i
f(a, b);     // arguments are separate

// COMMON BUG in trading message assembly:
msg.setPrice(nextPrice()).setQty(nextQty());
// nextPrice() and nextQty() calls: which runs first?
// In C++17: method call on msg runs before its argument,
// but the TWO chained calls' inner args are still unsequenced
```

Follow-up HRT may ask: What changed in C++17 for chained function calls like obj.f().g()?

HRT Insight: HRT code that builds messages with chained setters must not rely on evaluation order of arguments — two side-effectful calls in the same expression chain may run in any order.

Q13. Explain the C++ memory model's rules for out-of-thin-air (OOTA) values and why they matter for lock-free algorithms. [MM]

Answer

Out-of-thin-air (OOTA): a theoretical scenario in weakly-ordered models where a thread "reads" a value that no store has ever produced — arising from circular relaxed load/store reasoning. The C++ standard explicitly forbids OOTA values, but the formal model still allows certain "thin-air" reads in theory. Practically: on x86 (TSO), ARM (weakly ordered), and POWER, real hardware does not produce OOTA values, but compiler optimisations theoretically could under relaxed ordering. The consequence: any use of memory_order_relaxed in a dependency chain must be verified against both the hardware and the compiler's transformation model.

Code

```
// OOTA scenario (theoretical, not real hardware):
// Thread 1: r1 = x.load(relaxed); y.store(r1, relaxed);
// Thread 2: r2 = y.load(relaxed); x.store(r2, relaxed);
// Formally allows r1=r2=42 even though no thread stored 42
// C++ standard says this is forbidden (anti-OOTA guarantee)

// PRACTICAL: relaxed is safe for counters (no dependency)
std::atomic<int> counter{0};
counter.fetch_add(1, std::memory_order_relaxed); // safe
```

```
// DANGEROUS: relaxed load feeding into a pointer dereference
std::atomic<Node*> head{nullptr};
Node* p = head.load(std::memory_order_relaxed);
if (p) p->value;  // MAY READ stale head, then stale value!
// Need acquire for the load:
Node* p2 = head.load(std::memory_order_acquire);
if (p2) p2->value; // SAFE: acquire sees all stores before release

// HRT pattern: publication protocol
struct Snapshot { int price; int qty; };
std::atomic<Snapshot*> published{nullptr};

void publish(Snapshot* s) {
    published.store(s, std::memory_order_release);
}
Snapshot* read() {
    return published.load(std::memory_order_acquire);
    // Acquire: if non-null, all fields of *s are visible
}
```

Follow-up HRT may ask: Why is consume ordering (memory_order_consume) rarely used despite being weaker than acquire?

HRT Insight: HRT's order book snapshot publication is exactly this pattern: release store of the pointer, acquire load on the consumer side. Using relaxed for the pointer load would be a latent race on ARM.

TOPIC 4 — STL Internals & Performance

Q14. How does std::vector resize? Explain amortised O(1) push_back and why the growth factor matters. [DS]

Answer

When push_back exceeds capacity, vector allocates a new buffer (typically 1.5x or 2x current capacity), copies/moves all elements, then frees the old buffer. Over N push_backs: with 2x growth, total copies = 1+2+4+...+N = 2N = O(N), so amortised O(1) per push_back. Growth factor choice: 2x means higher peak memory (current + new buffer both live); 1.5x reduces peak but slightly more reallocations. MSVC uses 1.5x, GCC uses 2x. For latency-critical code: reserve() to prevent any reallocation on the hot path.

Code

```
std::vector<Order> orders;
orders.reserve(10000); // single allocation, no reallocs

// Without reserve: reallocs at 1,2,4,8,16,...
// Each realloc: O(n) move of all elements
// With reserve(N): guaranteed 0 reallocs for first N pushes

// Emplace vs push_back:
orders.emplace_back(price, qty, side); // construct in-place, no move
orders.push_back(Order{price,qty,side}); // construct temporary, then move

// Data layout: contiguous in memory
Order* raw = orders.data(); // pointer to first element
// Cache-friendly: prefetcher can load ahead

// When vector reallocates: move constructor must be noexcept
// else falls back to copy (O(n) copies instead of O(n) moves)
struct Order {
```

```
    Order(Order&&) noexcept = default; // crucial for performance
};
static_assert(std::is_nothrow_move_constructible_v<Order>);

// shrink_to_fit: release excess capacity
orders.shrink_to_fit(); // may reallocate to fit exactly

// swap trick (pre-C++11 shrink):
// std::vector<Order>(orders).swap(orders);
```

Follow-up HRT may ask: What happens if you push_back an element that is a reference into the same vector?

HRT Insight: HRT order queues are vectors pre-reserved at startup to capacity of the maximum expected order count — no reallocation occurs on the trading path, guaranteeing O(1) push_back latency.

Q15. Explain std::deque's internal structure. Why is it NOT cache-friendly and when should you still use it? [DS]

▶ Answer

std::deque is a sequence of fixed-size chunks (typically 512 bytes each) managed through a map array of chunk pointers. Random access requires: look up chunk pointer in map, compute offset within chunk — two indirections vs vector's one. Not cache-friendly: consecutive elements may be in different chunks (different cache lines). Iterator invalidation: front/back insert/erase does not invalidate iterators; middle insert does. Use deque when: O(1) push_front is needed (queue with front pop), and you cannot use a ring buffer.

⌨ Code

```
// deque internals (conceptual)
// map: [chunk0*, chunk1*, chunk2*, chunk3*]
// chunk0: [e0, e1, ..., e511] (512 bytes)
// chunk1: [e512, e513, ..., e1023]

// Random access: 2 indirections
deque[768]; // -> map[1][768-512] = chunk1[256]

// push_front: O(1) -- no element moves!
std::deque<int> dq;
dq.push_front(1); // fast, no realloc of existing elements
dq.push_back(2);  // also fast

// Iterator stability:
auto it = dq.begin();
dq.push_front(99); // it may be invalidated (map reallocated)
dq.push_back(99);  // it may be invalidated
// push_back/front CAN invalidate all iterators if map resizes

// Use case: work-stealing deque (push/pop front for owner)
// Owner: push_front, pop_front (LIFO = cache warm)
// Stealer: pop_back (takes oldest work)

// Prefer ring buffer for fixed-capacity queues:
// - Single allocation, contiguous memory, cache-friendly
// - No pointer indirection
// - Fixed latency (no realloc)
```

Follow-up HRT may ask: How does std::deque compare to a circular buffer for a fixed-size queue?

HRT Insight: HRT uses ring buffers rather than deque for inter-thread queues — the two-level indirection of deque adds unpredictable cache misses on a hot path that must be <100ns.

Q16. How does std::unordered_map handle collisions and when does lookup degrade to O(n)? How do you write a good hash function? [DS]

Answer

std::unordered_map uses separate chaining: each bucket holds a singly-linked list of entries with the same hash. Average O(1) lookup; worst-case O(n) when all entries hash to the same bucket (e.g., adversarial input with a predictable hash). Good hash function properties: uniform distribution, fast computation, avalanche effect (one input bit change flips ~half output bits), and seed randomisation to prevent DoS. For small integer keys (instrument IDs), identity hash can actually be good; for strings, FNV-1a or wyhash are standard.

Code

```
// DEFAULT hash for int: usually identity -> poor for dense IDs
std::unordered_map<int,Order> byId;
// If keys are 0,1,2,...,N: identity hash has 0 collisions
// If keys are all multiples of bucket_count: all collide!

// GOOD: FNV-1a hash for arbitrary data
struct FNV1aHash {
    size_t operator()(uint64_t key) const noexcept {
        constexpr uint64_t basis = 14695981039346656037ULL;
        constexpr uint64_t prime = 1099511628211ULL;
        uint64_t h = basis;
        auto* p = reinterpret_cast<const uint8_t*>(&key);
        for (size_t i = 0; i < sizeof(key); ++i)
            h = (h ^ p[i]) * prime;
        return h;
    }
};

// BETTER: wyhash (extremely fast, excellent distribution)
// Use it for symbol strings in production

// Avoid worst case: reserve and set max_load_factor
std::unordered_map<int,Order> safe;
safe.reserve(10000);        // pre-size: no rehash
safe.max_load_factor(0.5);  // lower = fewer collisions, more memory

// Custom hash for fixed-length symbol (e.g., "AAPL\0\0\0\0")
struct SymbolHash {
    size_t operator()(const char* s) const noexcept {
        uint64_t h; std::memcpy(&h, s, 8); // read 8 bytes as uint64
        return std::hash<uint64_t>{}(h);
    }
};
```

Follow-up HRT may ask: What is open addressing vs chaining? Which does the STL use and why?

HRT Insight: HRT instrument lookup (symbol -> order book) uses a custom hash map with a fixed symbol length hash — the standard string hash is much slower for 4–8 character tickers.

Q17. What is iterator invalidation? Give the complete rules for vector, deque, map, and unordered_map. [DS]

Answer

Vector: any insertion that causes reallocation invalidates ALL iterators, pointers, and references. Insertion/deletion at a position invalidates iterators at that position and after (but not before). Deque: front/back insert may invalidate all iterators; middle insert always invalidates all. Map/set: insertion never invalidates existing iterators; erasure only invalidates the erased element's iterator. Unordered_map/set: insertion that causes rehash invalidates ALL iterators; erasure only invalidates the erased element. These rules are critical for code that caches iterators to map entries while the map is being modified.

Code

```
// VECTOR invalidation:
```

```
std::vector<int> v = {1,2,3,4,5};
auto it = v.begin() + 2; // points to 3

v.push_back(6);  // may reallocate -> it INVALID
// FIX: store index, not iterator
size_t idx = 2; // always valid to use v[idx]

v.erase(v.begin()); // erases 1 -> it (was 3) is now at pos 1
// it points to 2 now -- value changed!

// MAP: stable iterators!
std::map<int,std::string> m = {{1,"a"},{2,"b"},{3,"c"}};
auto mit = m.find(2); // points to {2,"b"}
m.insert({4,"d"});    // mit STILL valid
m.erase(1);           // mit STILL valid (erased different elem)
m.erase(mit);         // NOW mit is invalid

// UNORDERED_MAP: rehash invalidates all
std::unordered_map<int,Order> um;
um.reserve(100); // prevent rehash for first 100 inserts
auto uit = um.find(1);
um[200] = Order{}; // may rehash if > 100 elements -> uit invalid

// SAFE pattern: re-find after modification
modifyMap(um);
auto safe_it = um.find(key); // always re-find
```

Follow-up HRT may ask: How does node extraction (C++17) affect iterator validity?

HRT Insight: HRT order books cache iterators to price levels for O(1) update. If the underlying container is accidentally a vector (invalidated by insertion) instead of map (stable), order book corruption is silent.

TOPIC 5 — Concurrency Deep Dives

Q18. What is the difference between std::mutex, std::timed_mutex, std::shared_mutex, and std::recursive_mutex? When would you choose each? [Con]

Answer

std::mutex: non-recursive, non-timed, exclusive — default choice. std::timed_mutex: adds try_lock_for/try_lock_until — use when you need timeouts to avoid deadlock (e.g., try to acquire, if not available within 1us, take a different path). std::shared_mutex: reader-writer lock — multiple simultaneous readers, exclusive writer — use for read-heavy data like configuration or order book snapshots. std::recursive_mutex: same thread can lock multiple times — signals a design problem; prefer refactoring to non-recursive. shared_mutex has higher overhead than mutex even for exclusive access.

Code

```
// shared_mutex: read-heavy order book snapshot
class SnapshotStore {
    mutable std::shared_mutex mtx_;
    BookSnapshot snapshot_;
public:
    // Many readers simultaneously:
    BookSnapshot read() const {
        std::shared_lock lk(mtx_); // shared: multiple OK
        return snapshot_;
    }
    // Only one writer at a time:
    void update(BookSnapshot s) {
```

```
        std::unique_lock lk(mtx_); // exclusive
        snapshot_ = std::move(s);
    }
};

// timed_mutex: avoid deadlock with timeout
std::timed_mutex tmtx;
void tryWork() {
    using namespace std::chrono_literals;
    if (tmtx.try_lock_for(100us)) {
        std::lock_guard lk(tmtx, std::adopt_lock);
        doWork();
    } else {
        // Timeout: take alternative path
        handleContention();
    }
}

// AVOID recursive_mutex: redesign instead
// If you need it: split locked and unlocked implementations
class Safe {
    std::mutex mtx_;
    void innerImpl() { /* no lock assumed held */ }
public:
    void outer() { std::lock_guard lk(mtx_); innerImpl(); }
    void inner() { std::lock_guard lk(mtx_); innerImpl(); }
};
```

Follow-up HRT may ask: Why is shared_mutex not always faster than mutex for read-heavy workloads?

HRT Insight: HRT uses shared_mutex for the risk parameter store (read thousands of times per second by many threads, written rarely by a config thread). Plain mutex would serialise all reads.

Q19. What is the ABA problem in lock-free programming? Walk through a concrete example and its solutions. [Con]

▶ Answer

ABA problem: thread T1 reads a value A from an atomic location. T2 changes it from A to B and back to A. T1's compare_exchange_strong sees A and succeeds — but the world has changed. Classic case in lock-free stack: T1 reads head=A (next=B). T2 pops A and B, then pushes a new node at the same address (recycled memory) as A. T1 pops A, setting head=B — which was freed. Solutions: tagged pointers (version counter in high bits), hazard pointers, epoch-based reclamation, or avoiding memory reclamation on the hot path entirely.

⌨ Code

```
// ABA IN LOCK-FREE STACK:
// Stack: A -> B -> C
// T1: reads head=A, next(A)=B
// T2: pops A, pops B, recycles A's memory, pushes A again
// Stack is now: A -> C
// T1: CAS(head, A, B) SUCCEEDS -- but B is freed!

// FIX 1: tagged pointer (version counter)
struct TaggedPtr {
    Node* ptr;
    uintptr_t tag; // increment on every successful CAS
};
std::atomic<TaggedPtr> head;

void push(Node* n) {
    TaggedPtr old = head.load(std::memory_order_acquire);
    TaggedPtr next;
    do {
        n->next = old.ptr;
        next = {n, old.tag + 1}; // new tag: unique combination
```

```
    } while (!head.compare_exchange_weak(old, next,
             std::memory_order_release,
             std::memory_order_relaxed));
}

// FIX 2: never recycle -- pre-allocate, never free
// (valid for bounded-lifetime systems)

// FIX 3: epoch-based reclamation
// Only free when no thread is in an old epoch

// x86_64: 48-bit virtual address -> use upper 16 bits for tag
// (but platform-dependent!)
```

Follow-up HRT may ask: How does the Linux kernel solve the ABA problem in its lock-free data structures?

HRT Insight: HRT's lock-free message pool uses tagged pointers to prevent ABA when nodes are recycled back to the pool while another thread is still observing the head pointer.

Q20. What is the happens-before relation for thread creation, joining, mutex lock/unlock, and atomic release/acquire? [Con]

Answer

The key synchronizes-with pairs that establish happens-before: (1) Thread::start() synchronizes-with the first action in the new thread — all writes before start() are visible in the new thread. (2) Thread exit synchronizes-with join() returning — all writes in the thread are visible after join(). (3) Mutex unlock synchronizes-with the next lock() of the same mutex — all writes under the old lock are visible under the new lock. (4) Atomic release store synchronizes-with acquire load of the same atomic. These compose transitively — enabling reasoning about complex multi-threaded protocols.

Code

```
// (1) Thread start: writes before start() visible in thread
int config = 42;
std::thread t([&]{ std::cout << config; }); // sees 42
config = 99; // AFTER start: MAY or MAY NOT be seen by t
t.join();

// (2) Thread join: all writes in t visible after join
int result = 0;
std::thread t2([&]{ result = computeHeavy(); });
t2.join();
std::cout << result; // SAFE: happens-before via join

// (3) Mutex: unlock-HB-lock chain
int shared = 0;
std::mutex m;
// Thread A:
{ std::lock_guard lk(m); shared = 100; } // unlock
// Thread B (after A's unlock):
{ std::lock_guard lk(m); std::cout << shared; } // sees 100

// (4) Release-Acquire: synchronizes-with
std::atomic<int*> ptr{nullptr};
int data = 99;
// Producer:
data = 42;                                // (a)
ptr.store(&data, memory_order_release); // (b) sync-w (c)
// Consumer:
int* p = ptr.load(memory_order_acquire); // (c)
if (p) assert(*p == 42); // SAFE: (a) HB (b) SW (c) HB this
```

Follow-up HRT may ask: Does a mutex guarantee visibility of ALL memory written before the lock, or only memory written while holding the same mutex?

HRT Insight: The answer to the follow-up is ALL memory (any write before unlock is visible after lock), not just writes under the same mutex — this is the basis for safe "lock-free read, locked write" patterns at HRT.

Q21. What is false sharing and how do you detect it? What is the performance impact on a dual-core trading system? [Con]

Answer

False sharing: two CPU cores modify different variables that share the same 64-byte cache line. Each modification causes a MESI coherence broadcast, forcing the other core to invalidate its cache line and reload — costing ~200 clock cycles per modification. On a dual-core 3GHz system: false-shared atomics can sustain only ~15M operations/sec vs ~3B for non-shared. Detection: perf stat -e cache-misses,L1-dcache-load-misses, or Intel VTune. Fix: pad with alignas(64) to separate hot variables onto different cache lines.

Code

```
// BEFORE: false sharing -- both in same 64-byte cache line
struct Statistics {
    std::atomic<uint64_t> feedMsgs{0};    // bytes 0-7
    std::atomic<uint64_t> bookUpdates{0}; // bytes 8-15
    std::atomic<uint64_t> orders{0};      // bytes 16-23
    // All in ONE cache line!
};
// Thread 1 increments feedMsgs -> invalidates Thread 2's cache
// Thread 2 increments bookUpdates -> invalidates Thread 1's cache

// AFTER: each counter on its own cache line
struct alignas(64) Counter {
    std::atomic<uint64_t> value{0};
    // 56 bytes padding implied by alignas(64)
};
struct Statistics2 {
    Counter feedMsgs;    // cache line 0
    Counter bookUpdates; // cache line 1
    Counter orders;      // cache line 2
};

// Detect with perf:
// perf stat -e L1-dcache-load-misses,LLC-load-misses ./trading_app

// Verify alignment:
static_assert(offsetof(Statistics2, bookUpdates) == 64);
static_assert(sizeof(Counter) == 64);
```

Follow-up HRT may ask: What is the cache line size on modern Intel vs ARM CPUs?

HRT Insight: On HRT's trading servers, false sharing between the feed handler's sequence counter and the book thread's position index caused 10x throughput degradation — detected with perf and fixed with alignas(64).

TOPIC 6 — Design, Idioms & Systems Thinking

Q22. What is the Pimpl idiom? Walk through a complete implementation including move semantics and binary compatibility. [MM]

Answer

Pimpl (pointer to implementation) hides implementation details behind a forward-declared opaque pointer. Benefits: (1) ABI stability — changing impl details does not change the class layout, so clients need not recompile. (2) Faster compilation — implementation headers not included by clients. (3) Clean interface. Requirements for correct Pimpl with unique_ptr: destructor must be defined in the .cpp file where Impl is complete; move constructor/assignment must be declared and defined in .cpp too.

Code

```
// --- orderbook.h (public header, no impl details) ---
class OrderBook {
    struct Impl;                          // forward declare
    std::unique_ptr<Impl> pImpl_;
public:
    explicit OrderBook(std::string symbol);
    ~OrderBook();                         // MUST declare
    OrderBook(OrderBook&&) noexcept;      // MUST declare
    OrderBook& operator=(OrderBook&&) noexcept;
    OrderBook(const OrderBook&) = delete;
    OrderBook& operator=(const OrderBook&) = delete;
    void addOrder(int64_t price, int32_t qty, char side);
    int64_t bestBid() const;
};

// --- orderbook.cpp (impl details hidden here) ---
#include "orderbook.h"
#include <map>  // only in .cpp, not leaked to clients

struct OrderBook::Impl {
    std::string symbol;
    std::map<int64_t, int32_t, std::greater<int64_t>> bids;
    std::map<int64_t, int32_t> asks;
};

OrderBook::OrderBook(std::string s)
    : pImpl_(std::make_unique<Impl>(std::move(s))) {}

OrderBook::~OrderBook() = default;  // Impl complete here
OrderBook::OrderBook(OrderBook&&) noexcept = default;
OrderBook& OrderBook::operator=(OrderBook&&) noexcept = default;

void OrderBook::addOrder(int64_t p, int32_t q, char s) {
    if (s == 'B') pImpl_->bids[p] += q;
    else          pImpl_->asks[p] += q;
}
int64_t OrderBook::bestBid() const {
    return pImpl_->bids.empty() ? 0 : pImpl_->bids.begin()->first;
}
```

Follow-up HRT may ask: How does Pimpl interact with inline functions and devirtualisation optimisations?

HRT Insight: HRT's public API headers use Pimpl extensively — changing internal data structures (e.g., switching from map to a sorted vector) does not require client recompilation, enabling rolling library updates.

Q23. Explain std::optional, std::variant, and std::any — their memory layouts, performance characteristics, and when to choose each. [MM]

Answer

optional<T>: stack-allocated, sizeof = sizeof(T) + 1 (plus alignment), no heap, zero overhead for access when engaged. Use for nullable value types. variant<Ts...>: stack-allocated, sizeof = max(sizeof(Ts)) + discriminant, no heap, std::visit generates jump table (O(1)). Use for closed type sets (message types, state machines). any: may heap-allocate for large types (SBO threshold ~32 bytes), type-erased (no compile-time type info), access via any_cast is O(1) but throws on wrong type. Use only when the type set is truly open. For trading messages: variant wins — zero allocation, exhaustive switch, compile-time safety.

Code

```
// optional: nullable price (0 is a valid price!)
std::optional<int64_t> bestBid() const {
    if (bids_.empty()) return std::nullopt;
    return bids_.begin()->first;
}
auto b = book.bestBid();
if (b) process(*b); // explicit null check

// sizeof optional<int64_t> = 16 (8 data + 1 flag + 7 pad)
static_assert(sizeof(std::optional<int64_t>) == 16);

// variant: closed message set (no heap!)
using Message = std::variant<AddOrder, CancelOrder, Trade, Quote>;

// std::visit: compiler generates switch/jump table
void dispatch(const Message& m, OrderBook& book) {
    std::visit(overloaded{
        [&](const AddOrder& a)    { book.add(a); },
        [&](const CancelOrder& c) { book.cancel(c.id); },
        [&](const Trade& t)       { book.onTrade(t); },
        [&](const Quote& q)       { book.onQuote(q); },
    }, m);
}

// overloaded helper (C++17)
template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
template<class... Ts> overloaded(Ts...) -> overloaded<Ts...>;

// any: avoid in hot paths (heap alloc, type erasure)
std::any val = 42;
int x = std::any_cast<int>(val); // throws bad_any_cast if wrong type
```

Follow-up HRT may ask: What is the overhead of std::visit with N types vs a vtable with N virtual functions?

HRT Insight: HRT's message processing pipeline uses variant<> for market data messages — the compiler reduces std::visit to a computed goto (jump table), matching the performance of hand-written switch statements.

Q24. What is expression templates and how are they used to eliminate temporary objects in mathematical expressions? [MM]

Answer

Expression templates: a technique where overloaded operators return proxy objects (expression nodes) rather than computed values. The final result is only computed when assigned to a concrete type, allowing the compiler to fuse multiple operations into a single loop — eliminating intermediate temporaries. Used in Eigen, Blaze, and other linear algebra libraries. For trading, expression templates can eliminate temporaries in fixed-point price arithmetic, e.g., a*b + c*d without allocating intermediate vectors.

Code

```
// WITHOUT expression templates: two temporaries
// Vector c = a + b; // temp1 = a+b
// Vector d = c * e; // temp2 = c*e, then d = temp2

// WITH expression templates: one pass
template<typename L, typename R>
struct AddExpr {
    const L& l; const R& r;
    double operator[](size_t i) const { return l[i] + r[i]; }
    size_t size() const { return l.size(); }
};

struct Vec {
    std::vector<double> data;
```

```
    double operator[](size_t i) const { return data[i]; }
    size_t size() const { return data.size(); }
    // Assign from any expression:
    template<typename Expr>
    Vec& operator=(const Expr& e) {
        data.resize(e.size());
        for (size_t i = 0; i < e.size(); ++i)
            data[i] = e[i]; // evaluate lazily
        return *this;
    }
};

template<typename L, typename R>
AddExpr<L,R> operator+(const L& l, const R& r) {
    return {l, r}; // return proxy, no computation
}

Vec a{...}, b{...}, c{...}, result;
result = a + b + c;
// No temporaries: single loop computes a[i]+b[i]+c[i]
```

Follow-up HRT may ask: What are the compilation-time costs of expression templates? Why do some projects avoid them?

HRT Insight: HRT uses expression templates in its fixed-point math library for P&L calculation — eliminating temporaries in (price - avgCost) * qty * multiplier reduces cache pressure on the analytics path.

Q25. What is the difference between intrusive and non-intrusive containers? When would you use an intrusive list? [MM]

▶ Answer

Non-intrusive containers (std::list): store elements in separately allocated nodes that contain the element plus pointers. The element class does not know it is in a container. Overhead: one heap allocation per element, two pointer jumps for traversal. Intrusive containers (Boost.Intrusive, kernel lists): the element class contains the next/prev pointers as members. No separate node allocation — the element IS the node. Splice is O(1) with no allocation. Used when: O(1) element removal from any list the element is in, no extra allocation budget, or multiple containers share the same objects.

⌨ Code

```
// NON-INTRUSIVE: std::list
// Order -> list<Order>::node(next*, prev*, Order data)
// Two allocations: order + list node
// Cannot remove from list without having list reference

// INTRUSIVE: element knows it is in a container
struct Order {
    int64_t price;
    int32_t qty;
    // Intrusive list hook:
    Order* next = nullptr;
    Order* prev = nullptr;
};

class IntrusiveList {
    Order* head_ = nullptr;
    Order* tail_ = nullptr;
public:
    void push_back(Order* o) {
        o->prev = tail_;
        o->next = nullptr;
        if (tail_) tail_->next = o;
        else head_ = o;
        tail_ = o;
    }
```

```
    // O(1) remove: element knows its neighbours
    void remove(Order* o) {
        if (o->prev) o->prev->next = o->next;
        else head_ = o->next;
        if (o->next) o->next->prev = o->prev;
        else tail_ = o->prev;
    }
};

// Use case: order in multiple queues simultaneously
// An order can be in the price-level queue AND the time queue
// with TWO sets of intrusive pointers -- no extra allocation
```

Follow-up HRT may ask: What is the Boost.Intrusive library and how does it make intrusive containers safer?

HRT Insight: HRT order management uses intrusive lists for O(1) cancel — when an order is cancelled, it removes itself from the price level list without needing to search, and without allocating memory at cancel time.

Q26. What are the tradeoffs between inheritance-based polymorphism and policy-based design (template policies)? [MM]

Answer

Inheritance polymorphism: runtime dispatch via vtable, one pointer indirection per call, prevents inlining, enables heterogeneous containers, no compile-time type dependency. Policy-based design (templates): compile-time dispatch, zero overhead (fully inlined), separate compilation units required, no heterogeneous containers without type erasure, enables "pay for what you use". For hot-path trading code: prefer policy-based when the implementation is known at compile time; use runtime polymorphism only for plugin-style extensibility.

Code

```
// RUNTIME polymorphism: flexible, overhead on every call
struct ExecutionStrategy {
    virtual void onFill(int64_t price, int32_t qty) = 0;
    virtual ~ExecutionStrategy() = default;
};

class TWAP : public ExecutionStrategy {
public:
    void onFill(int64_t p, int32_t q) override { /* ... */ }
};

std::unique_ptr<ExecutionStrategy> strategy = std::make_unique<TWAP>();
strategy->onFill(price, qty); // vtable lookup every call

// POLICY-BASED: zero overhead, inlined
template<typename FillPolicy>
class OrderEngine {
    FillPolicy policy_;
public:
    void onFill(int64_t p, int32_t q) {
        policy_.onFill(p, q); // INLINED: no vtable
    }
};

struct TWAPPolicy {
    void onFill(int64_t p, int32_t q) { /* TWAP logic */ }
};
struct VWAPPolicy {
    void onFill(int64_t p, int32_t q) { /* VWAP logic */ }
};

OrderEngine<TWAPPolicy> twapEngine; // specific at compile time
OrderEngine<VWAPPolicy> vwapEngine; // different type, no overhead
```

```
// Hybrid: policy for hot path, virtual for config
// Inject policy at startup via template instantiation
```

Follow-up HRT may ask: How do you achieve runtime selection of policies (e.g., from a config file) without virtual dispatch?

HRT Insight: HRT execution algorithms are template-parameterised on their fill policy — the specific algorithm is determined at compile time per strategy, giving full inlining on the critical fill-processing path.

Q27. What is std::launder and when is it necessary? [MM]

Answer

std::launder is a pointer-laundering operation needed after placement new. The problem: the compiler may cache values of const or reference members through a pointer because it assumes objects do not change at a given address. After placement new replaces an object at the same address, the compiler may still use cached values from the old object. std::launder tells the compiler: "treat this pointer as if the object at this address was freshly created — do not use cached values." Required whenever you placement-new over a const-member-containing type.

Code

```
// WITHOUT launder: compiler may use cached value
struct Immutable {
    const int val;
    Immutable(int v) : val(v) {}
};

alignas(Immutable) std::byte buf[sizeof(Immutable)];
new (&buf) Immutable{42};
Immutable* p1 = reinterpret_cast<Immutable*>(&buf);
std::cout << p1->val; // 42 -- OK here

// Now replace with a new object at the same address:
new (&buf) Immutable{99};
Immutable* p2 = reinterpret_cast<Immutable*>(&buf);
// p2->val MAY print 42! Compiler cached val through p1/addr

// FIX: std::launder
Immutable* p3 = std::launder(reinterpret_cast<Immutable*>(&buf));
std::cout << p3->val; // guaranteed 99

// Also needed after placement-new into aligned_storage:
std::aligned_storage_t<sizeof(Order), alignof(Order)> storage;
new (&storage) Order{price, qty};
Order* o = std::launder(reinterpret_cast<Order*>(&storage));

// C++17 feature -- use in pool allocators and object caches
// Not needed if the type has no const members or references
```

Follow-up HRT may ask: Why was std::launder added in C++17? What bug was it fixing?

HRT Insight: HRT's object pools for Order objects use aligned_storage + placement new — std::launder ensures that when an order slot is reused, the compiler does not incorrectly serve stale cached field values from the previous order.

Q28. What are the rules for [[nodiscard]], [[likely]], [[unlikely]], and [[no_unique_address]] attributes? How do they affect code generation? [MM]

Answer

[[nodiscard]]: compiler warning if the return value is discarded — critical for error-handling functions that return error codes (forgetting to check is a bug). [[likely]]/[[unlikely]] (C++20): hints the branch predictor and code layout — the likely branch is placed to avoid a jump, reducing pipeline stalls. [[no_unique_address]] (C++20):

allows empty base class members to share address with following members, enabling zero-cost policy injection in template classes without EBO tricks.

Code

```
// [[nodiscard]]: enforce error checking
[[nodiscard]] bool sendOrder(const Order& o) {
    return socket_.send(o);
}

sendOrder(order);   // WARNING: nodiscard result ignored
if (!sendOrder(order)) handleFail(); // correct

// [[nodiscard]] with reason (C++20)
[[nodiscard("Check return: false means order rejected")]]
bool submitOrder(const Order& o);

// [[likely]]/[[unlikely]]: branch layout hints
void processMsg(const Msg& m) {
    if (m.type == MsgType::Trade) [[likely]] {
        // Hot path: no jump, sequential execution
        processTrade(m);
    } else [[unlikely]] {
        // Cold path: compiler places after hot path
        handleControl(m);
    }
}

// [[no_unique_address]]: zero-cost policy member
struct EmptyLogger {
    void log(const char*) {} // empty: no state
};

template<typename Logger = EmptyLogger>
class Engine {
    [[no_unique_address]] Logger logger_; // 0 bytes if empty!
    int price_;
};
// sizeof(Engine<EmptyLogger>) == sizeof(int)
// sizeof(Engine<RealLogger>)  == sizeof(int)+sizeof(RealLogger)
```

Follow-up HRT may ask: What is the difference between [[nodiscard]] on a function vs on a class?

HRT Insight: HRT places [[nodiscard]] on all order submission and cancellation functions — silently ignoring a rejection code (e.g., "position limit exceeded") would cause trading strategy to proceed as if the order was accepted.

Q29. What is the "rule of two" for const and reference member variables? What constraints do they impose? [MM]

Answer

A class with const or reference members: (1) Cannot be default-constructed (unless you provide a constructor that initialises them). (2) Cannot be copy-assigned (you cannot rebind a reference or reassign a const member). (3) Cannot be moved-from in a useful way (the moved-from object still has the const/reference binding). This makes such classes non-assignable — they cannot be stored in containers that require assignment (std::vector::resize, std::sort). The fix: use pointer members instead of references, or const_cast (dangerous), or redesign.

Code

```
// CLASS WITH REFERENCE MEMBER:
class View {
    const OrderBook& book_; // reference member
public:
    explicit View(const OrderBook& b) : book_(b) {}
    // Copy assignment DELETED by compiler
```

```
    // View v1{b1}, v2{b2}; v1 = v2; // ERROR: cannot rebind ref
};

// Cannot store in vector (resize needs assignment):
// std::vector<View> views;
// views.push_back(View{book}); // ERROR (no assignment)

// FIX: use pointer instead of reference
class SafeView {
    const OrderBook* book_; // pointer: can reassign
public:
    explicit SafeView(const OrderBook* b) : book_(b) {}
    // All special members auto-generated correctly
};
std::vector<SafeView> views; // OK

// CONST MEMBER: same problems
struct Config {
    const int maxOrders; // prevents assignment
    Config(int m) : maxOrders(m) {}
};
// std::vector<Config> cfgs; // push_back may need assignment

// FIX: make the whole object const instead
const Config cfg{1000}; // or store in unique_ptr
```

Follow-up HRT may ask: What is std::reference_wrapper and when does it solve the reference-in-container problem?

HRT Insight: HRT strategy objects hold references to shared risk limits and market data — switching to pointer members allows strategy vectors to be resorted by P&L without compiler errors.

Q30. How would you profile and measure latency in a C++ trading application? What are the pitfalls of using clock_gettime? [MM]

Answer

Measuring latency correctly is hard. clock_gettime(CLOCK_MONOTONIC): reads the OS clock — adds a syscall overhead (~150ns on Linux). Use rdtsc (timestamp counter) instead: 1 cycle read, but must account for frequency scaling and cross-core synchronisation. Pitfalls: the CPU may reorder reads/stores across the rdtsc (use rdtscp or memory fence). Frequency scaling (turbo boost, power states) makes cycle-to-nanosecond conversion variable. JIT warm-up and branch predictor training affect early measurements. Use a minimum of 1M iterations, discard outliers, and report percentiles (p50/p99/p999) rather than mean.

Code

```
#include <cstdint>
#include <x86intrin.h>

// Accurate: rdtsc with serialising fence
inline uint64_t rdtsc_start() {
    uint32_t lo, hi;
    asm volatile("lfence\n\t"  // prevent CPU reordering before
                 "rdtsc\n\t"
                 : "=a"(lo), "=d"(hi) :: "memory");
    return ((uint64_t)hi << 32) | lo;
}

inline uint64_t rdtsc_end() {
    uint32_t lo, hi;
    asm volatile("rdtscp\n\t"   // rdtscp: serialises after
                 "lfence"
                 : "=a"(lo), "=d"(hi) :: "memory", "%rcx");
    return ((uint64_t)hi << 32) | lo;
}
```

```
// Benchmark with percentiles
std::vector<uint64_t> samples;
samples.reserve(1000000);
for (int i = 0; i < 1000000; ++i) {
    uint64_t t0 = rdtsc_start();
    orderBook.processUpdate(update);
    uint64_t t1 = rdtsc_end();
    samples.push_back(t1 - t0);
}
std::sort(samples.begin(), samples.end());

// Frequency: measure once at startup
double ghz = 3.2; // or use calibration
auto ns = [&](uint64_t cycles) { return cycles / ghz; };

std::cout << "p50:  " << ns(samples[500000]) << "ns\n";
std::cout << "p99:  " << ns(samples[990000]) << "ns\n";
std::cout << "p999: " << ns(samples[999000]) << "ns\n";
```

Follow-up HRT may ask: What is the "observer effect" in latency measurement and how do you minimise it?

HRT Insight: HRT measures every critical path in nanoseconds with rdtsc — including the feed-to-book latency, book-to-signal latency, and signal-to-order latency. Missing even one outlier (p999) could mask a systematic issue.